Système d'exploitation : TD1

janvier 2026 — Pierre Rousselin

Exercice 1 : Mousse au chocolat

Voici une recette de mousse au chocolat.

Ingrédients et matériel

- une tablette de chocolat à dessert
- 6 œufs
- une pincée de sel
- un saladier pour la mousse
- un récipient pour monter les blancs en neige
- un batteur électrique sinon c'est fatigant
- une casserolle pour faire fondre le chocolat (de préférence au bain-marie)

Recette très détaillée

1. Casser la tablette de chocolat en carrés à mettre dans la casserolle. (durée : 2)
2. Faire fondre le chocolat. (durée : 10)
3. Pour chaque œuf, séparer le blanc du jaune, mettre le blanc dans le récipient et le jaune dans le saladier. (durée : 2 pour chaque œuf)
4. Mélanger les jaunes d'œuf avec le chocolat fondu. (durée : 2)
5. Ajouter une pincée de sel dans le récipient et utiliser le batteur pour monter les blancs en neige. (durée : 6)
6. Incorporer délicatement les blancs en neige dans le chocolat. (durée : 10)

1. Combien de temps prend le processus « mousse au chocolat » pour un seul cuisinier ?

2. Il y a 6 tâches, et même 11 si l'on compte une tâche par œuf à séparer en blanc et jaune. Quelles sont les dépendances entre les tâches (c'est-à-dire, pour chaque tâche, quelles sont les tâches qui doivent être terminées avant de pouvoir la commencer) ? Dessiner un diagramme de dépendance.

3. Chercher des moyens, avec autant de cuisiniers que vous voulez de rendre la recette parallèle, de sorte qu'elle prenne le moins de temps possible (on a faim !).

--- * ---

Correction de l'exercice 1 :

1. $2 + 10 + 2 \times 6 + 2 + 6 + 10 = 42$ (pas fait exprès).

2.
- La tâche 2 dépend de 1.
- La tâche 4 dépend des 6 sous-tâches de 3 et de la tâche 2.
- La tâche 5 dépend des 6 sous-tâches de 3.
- La tâche 6 dépend des tâches 4 et 5.

Le diagramme de dépendance n'est pas un arbre, mais un DAG (Directed Acyclic Graph. Le chemin le plus coûteux en terme de temps entre l'extrémité 6 et les « feuilles » (tâches sans dépendances) est 24 : fournit une borne inférieure pour la durée optimale de la recette en parallèle.

3. Il y a bien sûr des choix arbitraires à faire et plusieurs possibilités.

Un exemple avec 7 cuisiniers C_1, C_2, ..., C_7 :

- C_1, ..., C_6 séparent les blancs des jaunes.
- C_7 casse la tablette (tâche 1), puis fait fondre le chocolat (tâche 2) et *attend* que les blancs soient séparés des jaunes (dans ce cas précis, avec durée connue à l'avance, il n'a pas à attendre, mais dans la vraie vie des processus c'est une autre histoire).

— C_1 (par exemple), après avoir séparé le blanc du jaune de son œuf, attend que ses camarades C_2, ..., C_6 aient fini (et ils peuvent partir), puis bat les blancs en neige.
— Le dernier à finir entre C_1 et C_7 (ici ce sera C_7 mais encore une fois, on triche) termine la recette avec les tâches 5 et 6.

La durée est alors le temps de travail de C_7 (il travaille tout le temps, dès le début et termine la recette), c'est-à-dire $2+10+6+10 = 24$. Vu la borne inférieure donnée précédemment, cette solution est optimale en terme de durée.

--- * ---

Voici quelques rappels sur les fonctions `fork`, `getpid` et `getppid`.

```
#include <unistd> /* fork */
#include <sys/types.h> /* type pid_t */

/* Crée un nouveau processus en dupliquant le processus appelant. Retourne :
 * -1 en cas d'échec,
 * 0 pour le processus enfant,
 * un entier strictement positif pour le processus parent qui est le PID
 * (identificateur de processus) du processus enfant nouvellement créé. */
pid_t fork(void);
/* retourne le PID du processus appelant */
pid_t getpid(void);
/* retourne le PPID, c'est-à-dire le PID du parent du processus appelant */
pid_t getppid(void);
```

Exercice 2 :
On considère le programme suivant.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

void aff_proc(const char *ch);

int main(void)
{
        switch (fork()) {
        case -1:
                perror("fork 1");
                return 1;
        case 0:
                aff_proc("Enfant 1");
                break;
        default:
                aff_proc("Parent 1");
        }

        switch (fork()) {
        case -1:
                perror("fork 2");
                return 1;
        case 0:
                aff_proc("Enfant 2");
                break;
        default:
                aff_proc("Parent 2");
```

```
        }

        printf("Je suis le processus %d et j'ai terminé\n", getpid());
        return 0;
}

void aff_proc(const char *ch)
{
        printf("%s : PID : %d, PPID : %d\n", ch, getpid(), getppid());
}
```

1. Combien de processus sont créés ?
2. Donner le nombre d'occurrences de chaque chaîne `(Parent|Enfant) [12]`, sur la sortie du programme.
3. Donner l'arbre de création des processus.
4. Est-ce que l'ordre dans lequel les processus se terminent est fixé ?
5. Voici un rappel sur la fonction `wait`.

   ```
   #include <sys/types.h>
   #include <sys/wait.h>
   /* attend la terminaison d'un des processus enfants
    * retourne le PID de l'enfant qui a terminé (-1 en cas d'erreur, par exemple
    * s'il n'y a plus d'enfant à attendre), et met son statut de sortie à
    * l'adresse statut ou l'ignore si elle vaut NULL */
   pid_t wait(int *statut);
   ```

   Modifier le programme précédent de sorte que chaque parent attende la terminaison de tous ses enfants avant de se terminer lui-même. Chaque parent devra afficher, chaque fois qu'un de ses enfants se termine la chaîne suivante :
   `Je suis <pid> et mon enfant de pid <pid> est mort.` en remplaçant les chaînes entre chevron par les valeurs appropriées.

--- * ---

Correction de l'exercice 2 :

1. Le premier processus `fork` deux fois et son premier enfant `fork` une fois, il y a donc en tout 4 processus.

2. `Parent 1` est écrit par le premier processus. `Parent 2` est écrit par le premier processus et son premier enfant, donc deux fois. `Enfant 1` est écrit par le premier enfant du premier processus. `Enfant 2` est écrit par le deuxième enfant du premier processus et l'enfant de son premier enfant, donc deux fois.

3. Il n'y a aucun mécanisme de synchronisation (`wait`), donc non c'est totalement non déterministe, on ne peut pas prévoir l'ordre de terminaison.

4. La solution bête et méchante serait d'ajouter, avant le dernier affichage :

```
pid_t enfant;
while ((enfant = wait(NULL)) != -1)
    printf("Je suis %d et mon enfant de pid %d est mort.\n",
           getpid(), enfant);
```

Alors, les processus qui n'ont pas enfantés n'entrent de toute façon pas dans la boucle. Mais comme les parents se retrouveront de toute façon dans le cas `default` du second `switch`, on peut le mettre là aussi.

--- * ---

Exercice 3 : Plein de `fork`

On considère le programme suivant :

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#define NUM_ITER_DEFAUT 3
int main(int argc, char *argv[])
{
        int num_iter, i;
        num_iter = argc < 2 ? NUM_ITER_DEFAUT : atoi(argv[1]);
        for (i = 0; i < num_iter; ++i) {
                switch (fork()) {
                case -1:
                        perror("fork");
                        return 1;
                case 0:
                        printf("plop\n");
                        break;
                default:
                        switch (fork()) {
                        case -1:
                                perror("fork");
                                return 1;
                        case 0:
                                break;
                        default:
                                printf("hop\n");
                                return 0;
                        }
                }
        }
        printf("bloup\n");
        return 0;
}
```

1. Quelle est la valeur de `num_iter` si le programme est exécuté sans argument, c'est-à-dire ainsi :
   ```
   $ ./plein_de_fork
   ```
2. On suppose que le programme a été exécuté de la façon suivante :
   ```
   $ ./plein_de_fork 2
   ```
   Dessiner l'arbre de création des processus dans ce cas. Combien de fois sont écrites les chaînes `plop`, `hop` et `bloup` ?
3. En général, quel est, en fonction de l'argument donné par l'utilisateur, le nombre total de processus créés, le nombre de `plop`, `hop` et `bloup` qui sont affichés ?
4. (Bonus) Comment diable expliquer la chose suivante ?
   ```
   $ gcc -Wall -O0 plein_de_forks.c
   $ ./a.out 2 | grep plop | wc -l
   5
   ```

--- * ---

Correction de l'exercice 3 :

1. Dans ce cas, `argc` vaut 1 (attention, c'est un peu confus, `argc` n'est pas vraiment le nombre d'arguments mais le nombre de mots sur la ligne de commande). Dans l'opérateur ternaire, la valeur de l'expression `argc < 2` est 1 donc celle de toute l'expression est `NUM_ITER_DEFAUT`, soit 3.
2. Ici, `num_iter` vaut 2. Le premier processus forke deux fois, puis meurt après avoir affiché `hop`. Ses enfants forkent deux fois puis meurent après avoir affiché `hop`. Alors i vaut 2 et les 4 processus restant

meurent après avoir affiché `bloup`. À chaque itération, le premier des deux enfants créés affiche `plop`, il y en a donc 3.

3. Si n est l'argument donné au premier processus, l'arborescence de tous les processus créé est un arbre binaire parfait de hauteur n. Il y a donc en tout $2^{n+1} - 1$ processus créés. Les feuilles de cette arborescence sont les processus qui sortiront de la boucle et écriront `bloup`, donc il y en a 2^n. Les sommets internes meurent avant la fin de la boucle en affichant `hop`, donc il y en a $2^n - 1$. Enfin, les enfants gauche dans l'arbre sont ceux qui écrivent `plop`, donc il y en a aussi $2^n - 1$.

4. L'écriture sur le terminal est *line-buffered*, c'est-à-dire que le tampon de sortie est écrit (et vidé) à chaque `'\n'`. Donc le nombre de `hop`, `plop`, etc calculé précédemment est bien correct dans le cas de l'écriture dans le terminal. Mais l'écriture dans un fichier ou dans un *pipe* est *block-buffered*, le tampon n'est écrit que lorsqu'il est plein (ou à la mort du processus). Les enfants sont des copies conformes du parent donc héritent aussi d'une copie du tampon ! Ici l'enfant gauche transmet un `plop` à ses deux enfants, il y aura donc deux `plop` supplémentaires.

--- * ---